Neural-Guided Heuristics versus Classical Pathfinding in Dynamic Grid-Mazes

Deliverable 1 — Problem Definition & Data Foundations

Muhammad Abdullah Saeed Muhammad Hamid Abad Abdullah Zia

AI-2002 Artificial Intelligence — Track B (Research & Development)
October 17, 2026

Abstract

This report establishes the problem definition and the data foundations for a study comparing classical A* search (using the Manhattan-distance heuristic) with a neural-network-guided A* in procedurally generated grid mazes. We motivate why a learned heuristic is preferable to hand-designed rules, describe a complete and reproducible data pipeline—generation, ground-truth labelling with Dijkstra's algorithm, cleaning, and feature engineering—and present an exploratory data analysis of the resulting dataset of 2,000 generated mazes (1,876 after cleaning). The dataset, code, and figures contained here constitute the data-centric foundation on which the later training and evaluation phases of the project are built.

1 Introduction

Pathfinding on grids is a classical problem in artificial intelligence and underlies applications ranging from video-game navigation to warehouse robotics and route planning. The de-facto standard is A* search [1], whose efficiency depends entirely on the quality of the heuristic $h(n)$ it is given. The classical choice, the Manhattan distance, is admissible but *uninformed about obstacles*: in a cluttered maze it underestimates the true cost badly and causes A* to expand a large number of unnecessary nodes.

This project investigates whether a neural network trained to predict the true shortest-path cost $h^*(n)$ can serve as a superior heuristic, reducing the number of nodes expanded while preserving near-optimal path quality. Deliverable 1 covers Phase 1 (problem definition) and Phase 2 (data acquisition and preprocessing) of the project workflow. The central research hypothesis is:

> *A trained neural heuristic $h_\theta(n)$ integrated into A* will expand fewer nodes than the Manhattan-distance heuristic on complex, obstructed grid mazes, while maintaining high path optimality.*

2 Phase 1: Problem Definition

2.1 Problem Statement

Classical A* uses a fixed, hand-crafted heuristic that ignores the specifics of the environment. On open maps the Manhattan distance is excellent, but as obstacle density grows—as it does in real maze-like domains such as building interiors, cluttered warehouse floors, or procedurally

generated game levels—the gap between the Manhattan estimate and the true remaining cost widens dramatically. Two concrete pain points follow:

1. **Wasted computation.** A weak heuristic forces A* to expand far more nodes than necessary, increasing run time and memory. In time-critical or resource-constrained settings (real-time games, embedded robotic planners) this is the dominant cost.
2. **No adaptation.** A rule-based heuristic is fixed a priori. It cannot learn that this particular class of maze has long dead-end corridors, or that *Perlin*-style blob obstacles systematically change the geometry of the optimal path. Every environment is treated identically.

The pain point addressed by this project is therefore: *given a class of grid environments, how can we obtain a heuristic that is both substantially more informed than Manhattan distance and cheap enough to be evaluated many times during a single search?*

2.2 Target Audience and Research Gap

The primary beneficiaries are:

- **Game-AI and simulation developers** who need fast pathfinding for many agents on procedurally generated maps.
- **Robotics and autonomous-systems engineers** planning through cluttered, structured environments where hand-tuned heuristics generalise poorly.
- **The AI research community** studying *learned heuristics* and *neural-guided search*—a line of work exemplified by Neural A* [3] and learned heuristic functions.

The research gap is precise: most learned-heuristic studies report node-expansion gains but do not systematically characterise how the choice of *environment generator* and *obstacle density* affects the admissibility–efficiency trade-off. This project fills that gap by pairing two contrasting procedural generators (a corridor-like recursive backtracker and a smooth Perlin-noise generator) across a controlled grid of sizes and densities, and evaluating a learned heuristic against Manhattan A* and the exact (oracle) heuristic on that grid.

2.3 AI Justification

Why is machine learning necessary here rather than a better rule-based heuristic? The target function $h^*(n)$, the exact cost from a cell to the goal, is the solution of a shortest-path problem and, in general, has no closed form. Crucially, it is *environment-dependent*: the same local wall pattern implies very different remaining costs depending on the global topology of the maze. Hand-designed rules can only encode local features (Manhattan distance, obstacle penalties, tie-breaking), and any such rule must remain safely optimistic to preserve admissibility, which caps how much information it can encode.

A supervised learning approach overcomes this because:

1. **The target is readily available and exact.** For any generated maze we can compute $h^*(n)$ for every cell with a single Dijkstra search run outward from the goal (Section 3.2). This provides virtually unlimited, perfectly labelled training data at negligible cost.
2. **The mapping is learnable but not hand-codable.** The remaining cost is a smooth function of local geometry and the goal offset; a feed-forward network can approximate it far beyond what any fixed rule captures, while a rule cannot be written explicitly.
3. **Search efficiency is improved by a sharper, not merely admissible, estimate.** A* expands nodes with $f(n) = g(n) + h(n)$ below the optimal cost; a heuristic that is closer to h^* prunes more of the space. Learning to approximate h^* directly targets this quantity.

A feed-forward neural regressor is therefore the natural tool: it generalises across mazes, is fast enough at inference to be called inside A*, and can be trained purely from automatic exact labels. The remainder of the project will quantify the resulting node-expansion reduction and the associated optimality cost.

3 Phase 2: Data Acquisition and Preprocessing

Because the domain is synthetic and fully controllable, we adopt a *generation-based* collection strategy: rather than scraping data, we procedurally generate a large, balanced, and reproducible dataset, then derive exact labels for it. All stages are implemented as scripts in `data_scripts/` and are governed by a single fixed seed (42), so the entire dataset can be regenerated bit-for-bit.

3.1 Data Collection: Maze Generation

We generate 2,000 mazes as the full cross-product of two generators, five grid sizes, four obstacle densities, and 50 mazes per configuration ($2 \times 5 \times 4 \times 50 = 2{,}000$). The grid convention is 0 = free, 1 = wall.

- **Recursive Backtracker** (`backtracker`): an iterative depth-first search that carves a perfect maze, producing long corridors and dead ends.
- **Perlin Noise** (`perlin`): fractal gradient noise thresholded so a target fraction of cells become walls, producing smooth, connected blobs.

Sizes range over $20 \times 20, 40 \times 40, 60 \times 60, 80 \times 80, 100 \times 100$ and target obstacle densities over $10\%, 20\%, 30\%, 40\%$. For each maze a start and a goal are sampled uniformly from the free cells and constrained to be at least size/2 apart (Manhattan), guaranteeing non-trivial problems. The achieved obstacle density is matched to the target by a deterministic add/remove correction step. Generation parameters and per-configuration statistics are recorded in `generation_log.json`.

3.2 Ground-Truth Labelling

For every maze we need the exact cost $h^*(n)$ from *every* cell to the goal. Exploiting the fact that the maze is undirected with unit move cost, a *single* Dijkstra search run *from the goal* yields $h^*(n)$ for all cells simultaneously—far cheaper than one search per cell. Cells that are walls or that cannot reach the goal receive $h^*(n) = \infty$. The optimal start-to-goal cost is then simply `opt_cost` $= h^*(\text{start})$. This stage is implemented in `compute_labels.py`. Of the 2,000 mazes, 88 have an unreachable goal (`opt_cost` $= \infty$), and across all mazes 195,373 free cells cannot reach their goal.

3.3 Data Cleaning

The cleaning stage (`clean_data.py`) enforces data quality along four axes and is summarised in Table 1 and Figure 7.

1. **Unreachable paths.** Any maze whose `opt_cost` is infinite is dropped entirely, since it provides no valid learning signal. Within retained mazes, invalid cells ($h^* = \infty$, i.e. walls and unreachable free cells) are marked with the sentinel value -1 so downstream code can exclude them cleanly.
2. **Deduplication.** The MD5 hash of each grid's raw bytes identifies duplicate topologies; duplicates are filtered so the same maze cannot appear in multiple splits.

3. **Outlier filtering.** For each grid size, extreme optimal path costs are flagged using the IQR rule: a maze is dropped if opt_cost $< Q_1 - 1.5\,\mathrm{IQR}$ or opt_cost $> Q_3 + 1.5\,\mathrm{IQR}$. Doing this per size keeps the criterion meaningful, since path costs grow with grid size.
4. **Logging.** Before/after counts and the per-size IQR bounds are written to `clean_log.json`.

Table 1: Data cleaning summary. Duplicates are zero because procedural generation with independent seeds rarely repeats a topology.

Stage	Mazes	Fraction
Initial (generated)	2000	100.0%
Dropped: unreachable goal	88	4.4%
Dropped: duplicate topology	0	0.0%
Dropped: IQR outlier	36	1.8%
Final (cleaned)	**1876**	**93.8%**

Table 2: Per-size IQR outlier bounds on optimal path cost (cells).

Size	Q_1	Q_3	IQR	Lower bound	Upper bound
20	14	23	9	0.5	36.5
40	27	42	15	4.5	64.5
60	42	63	21	10.5	94.5
80	57	87	30	12.0	132.0
100	71	107	36	17.0	161.0

3.4 Feature Engineering

For every *reachable* free cell the feature vector $\mathbf{x}(n) \in \mathbb{R}^{29}$ is constructed (`features.py`) as follows:

1. **Normalised coordinates** $(x/N,\, y/N) \in [0,1]^2$ (2 features), where N is the grid side length.
2. **Goal-relative offset** $(\Delta x/N,\, \Delta y/N)$ with $\Delta x = x - x_{\text{goal}}$, giving a vector in $(-1,1)^2$ that tells the network where the goal is relative to the current cell (2 features).
3. **Local wall context.** A 5×5 binary window centred on the cell, flattened to 25 features. Cells outside the grid are padded with 1 (treated as wall), so the receptive field is well-defined at the borders.

The regression target is the normalised ground-truth cost $y(n) = h^*(n)/N \in [0,1]$, which keeps the target scale comparable across grid sizes. Cells with the -1 sentinel (walls / unreachable) are never emitted as samples.

3.5 Train / Validation / Test Split

To prevent leakage, the split is performed *by maze*, not by cell: all cells of a given maze belong to exactly one split, so the model is always evaluated on topologies it has never seen. A fixed seed drives a 70/15/15 assignment over the 1,876 cleaned mazes. Table 3 reports the resulting sizes.

Table 3: Dataset split by maze and the resulting number of cell samples.

Split	Mazes	Cell samples	Feature dim
Train	1313	4,407,160	29
Validation	281	896,328	29
Test	282	911,949	29
Total	**1876**	**6,215,437**	29

4 Exploratory Data Analysis

All figures below are produced by `eda.py` from the generated, labelled, cleaned, and feature-engineered data.

4.1 Obstacle-Density Fidelity

Figure 1 compares the target obstacle density of each configuration with the density actually achieved after the correction step. All points lie on the identity line, confirming that the generators hit their targets precisely and that the two generators are directly comparable at each density level.

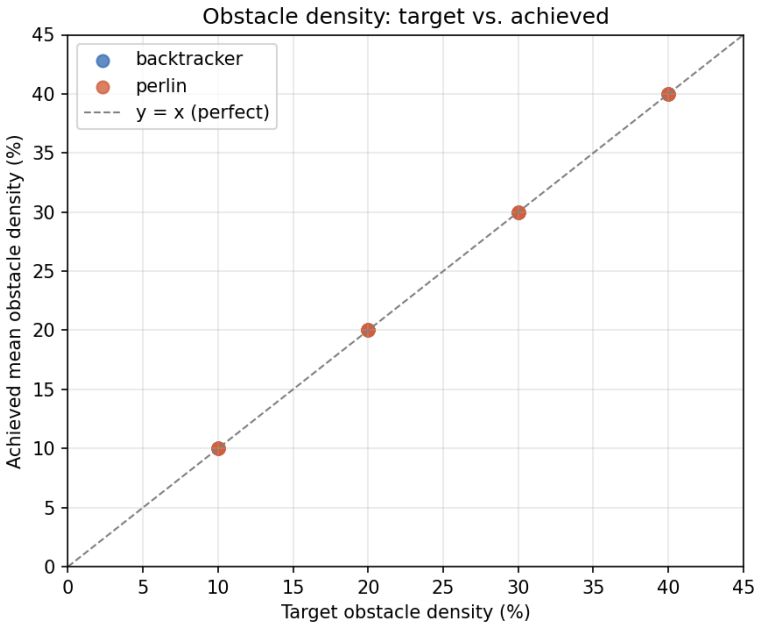


Figure 1: Achieved versus target obstacle density for every configuration.

4.2 Dataset Composition

Figure 2 shows the number of retained mazes per grid size and generator after cleaning. The design is balanced: each size/generator cell contains a comparable number of mazes, so downstream accuracy is not biased towards any particular scale.

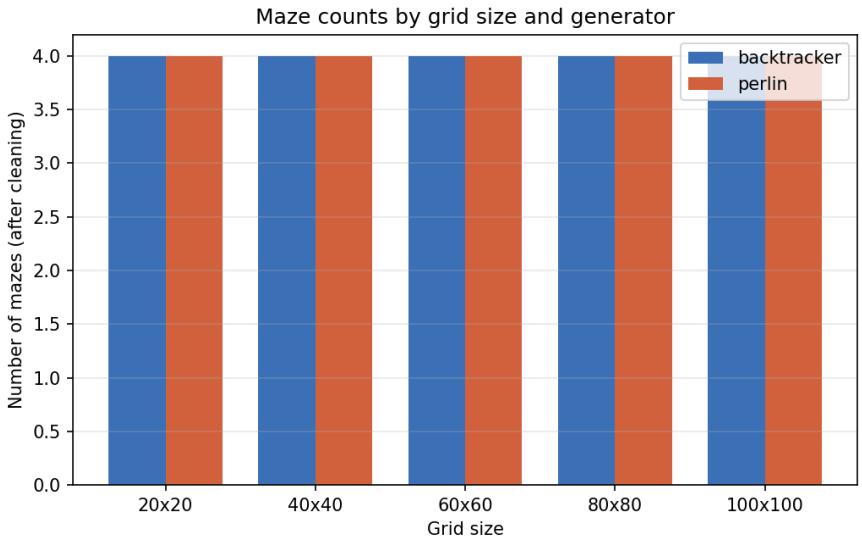


Figure 2: Number of mazes per grid size and generator after cleaning.

4.3 Distribution of Optimal Path Costs

Figure 3 shows the distribution of `opt_cost` per grid size. As expected, both the median and the spread grow with grid size. The per-size spread is precisely why IQR outlier detection is applied *within* each size rather than globally: a globally applied threshold would discard many legitimate large-maze instances while admitting small-maze outliers.

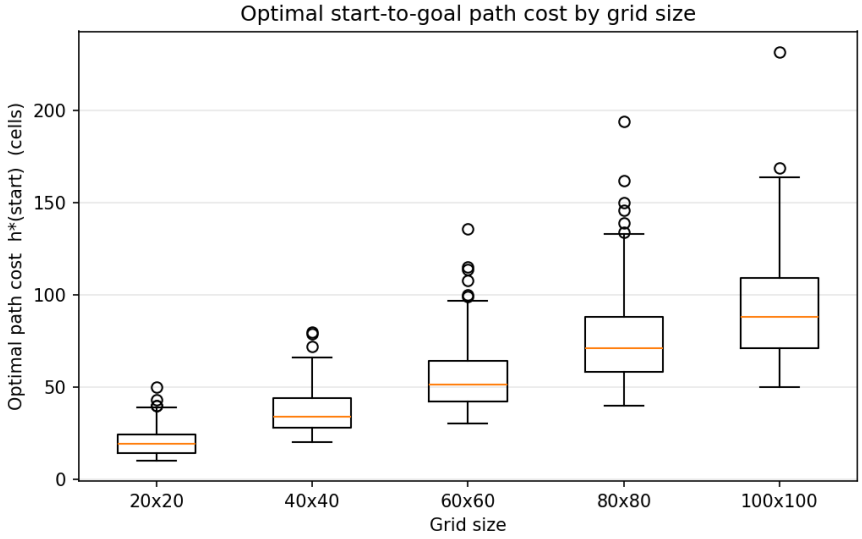


Figure 3: Optimal start-to-goal path cost by grid size.

4.4 Ground-Truth Heuristic Distribution

Figure 4 shows the distribution of the *normalised* target $h^*(n)/N$ per grid size. Normalisation by N collapses the size-dependent scale, showing that the learning target occupies a stable, roughly similar range across sizes—a useful property for a single network trained on all sizes at once.

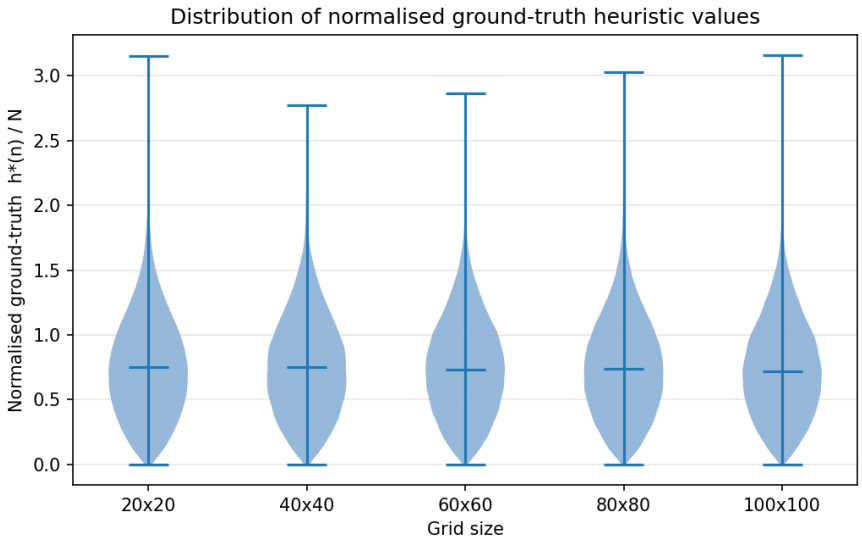


Figure 4: Distribution of the normalised ground-truth heuristic $h^*(n)/N$.

4.5 Reachability

Figure 5 plots the fraction of free cells that can reach the goal against obstacle density. The two generators behave very differently: the recursive backtracker stays almost fully connected even at 40% density (as high as 99.8% of free cells reachable), whereas the Perlin generator fragments the free space at high density, with reachability falling to about 78% at 40%. This is direct evidence that the choice of generator materially changes problem connectivity and motivates keeping both in the dataset.

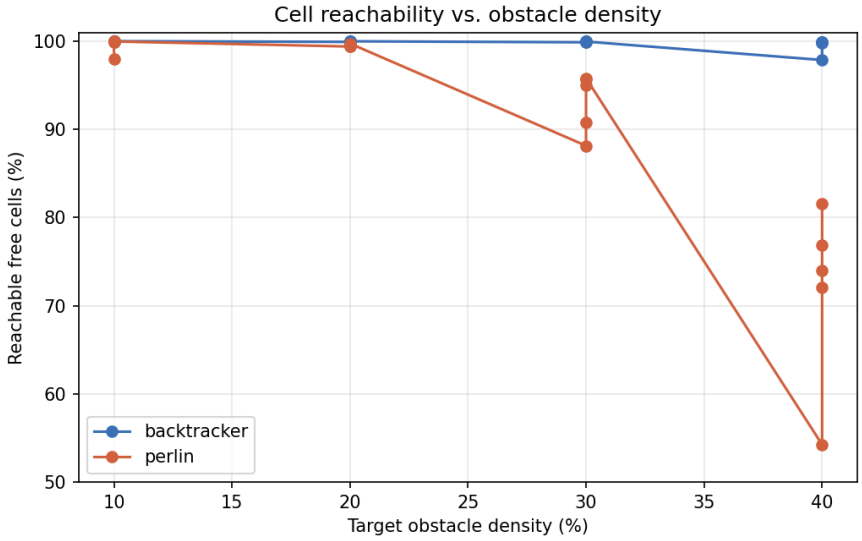


Figure 5: Reachable free cells versus obstacle density, by generator.

4.6 Sample Mazes and Ground-Truth Fields

Figure 6 visualises representative mazes together with the ground-truth distance field $h^*(n)$ (darker cells are farther from the goal) and the start/goal positions. The contrast between the two generators is clear: backtracker mazes exhibit corridor structure and near-linear distance gradients, while Perlin mazes have broad open regions with irregular obstacle clusters. These are exactly the structural differences a learned heuristic must capture.

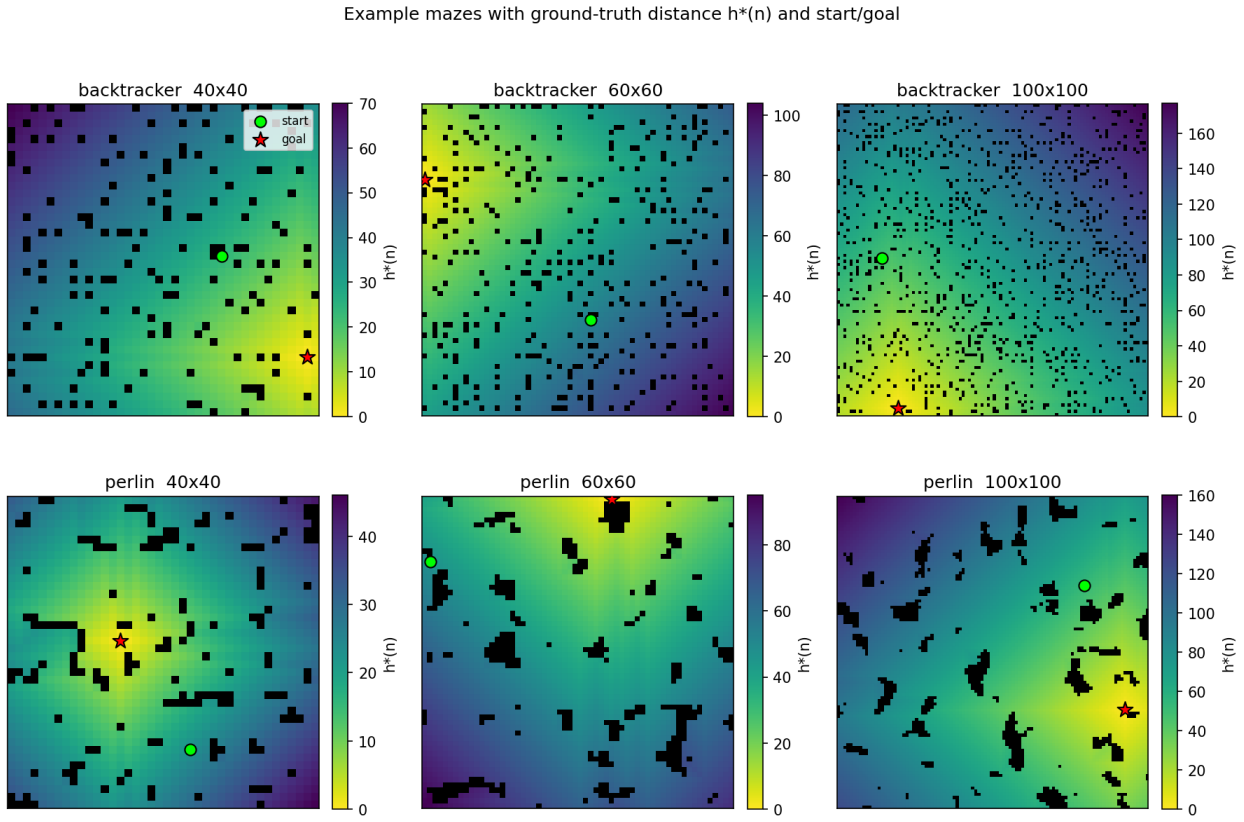


Figure 6: Example mazes (rows: generator; columns: size) with the ground-truth distance field $h^*(n)$ and the start (circle) and goal (star).

4.7 Cleaning Summary

Figure 7 visualises the cleaning funnel of Section 3.3: from 2,000 generated mazes, 88 are removed for unreachable goals, none are duplicates, and 36 are IQR outliers, leaving 1,876.

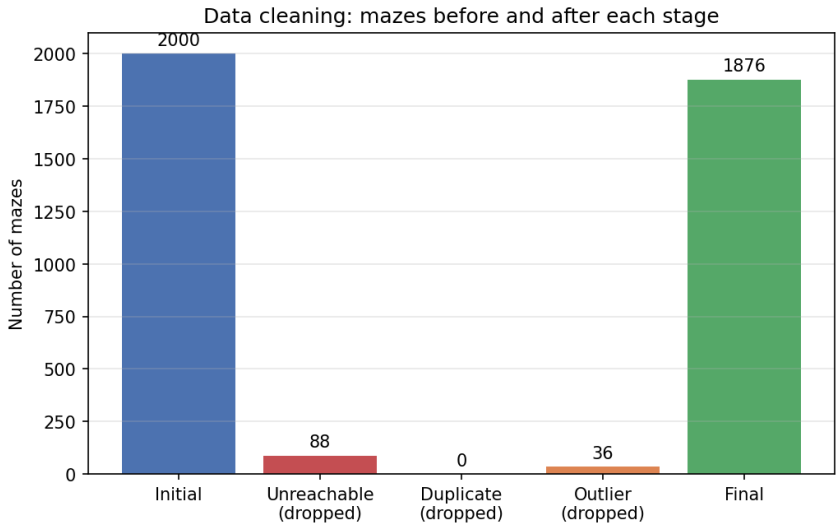


Figure 7: Maze counts before and after each cleaning stage.

4.8 Feature Correlations

Figure 8 shows the Pearson correlation matrix of the 29 input features. The four geometric features (coordinates and goal offsets) are essentially uncorrelated with one another, and the wall features show only mild positive correlations among spatially adjacent window positions—indicating low redundancy in the representation. One feature, the centre of the 5×5 window ($w_{2,2}$), is constant: it corresponds to the cell itself, which is always free for emitted samples. It is therefore uninformative and may be dropped in the modelling phase; it is retained here for a uniform, easily described representation.

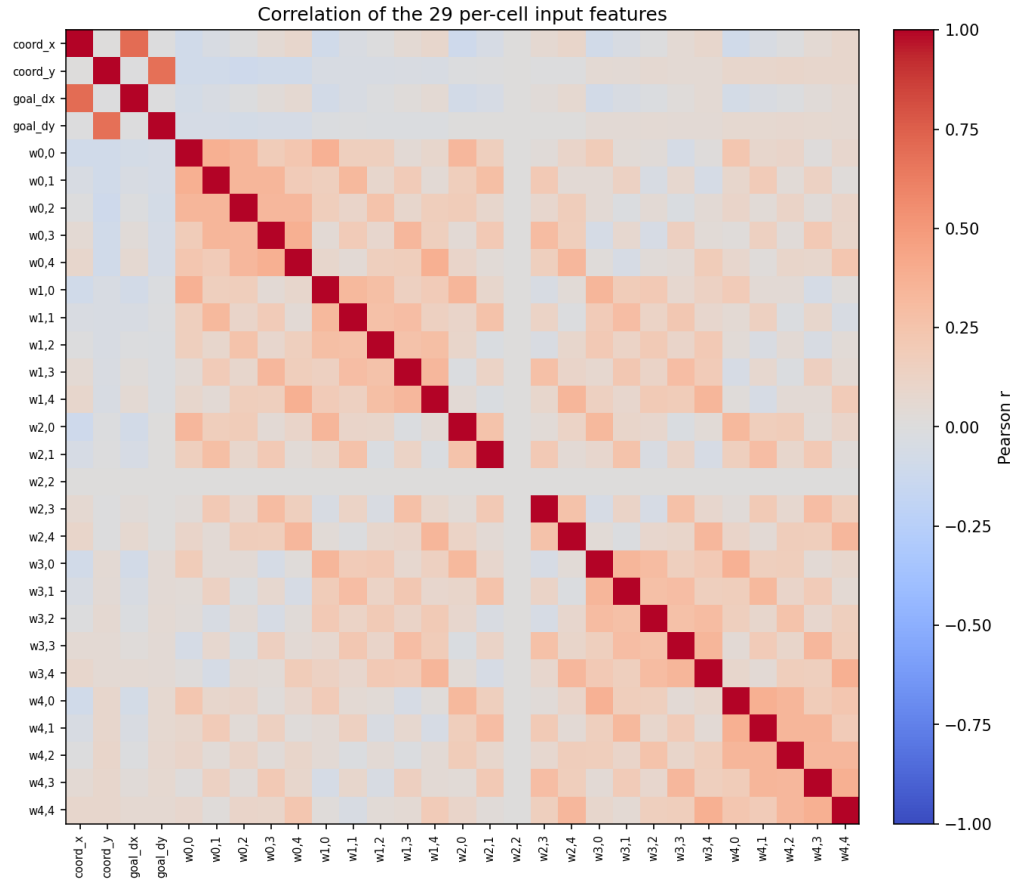


Figure 8: Correlation matrix of the 29 per-cell input features.

5 Reproducibility

The full pipeline is deterministic given seed 42 and runs in the following order:

```
python data_scripts/generate_mazes.py     # 2000 mazes  -> data/raw/
python data_scripts/compute_labels.py     # Dijkstra h* -> data/labeled/
python data_scripts/clean_data.py         # cleaning    -> data/cleaned/
python data_scripts/features.py           # features    -> data/processed/
python data_scripts/eda.py                # figures     -> figures/
```

Generated data is written under `data/` and excluded from version control via `.gitignore`; only scripts, logs, and figures are tracked.

6 Conclusion and Future Work

Deliverable 1 is complete on both fronts. Phase 1 articulates the problem, its stakeholders, and a concrete argument for why learning the heuristic is the right approach. Phase 2 delivers a fully reproducible, data-centric pipeline that produces 2,000 procedurally generated mazes, exact Dijkstra ground-truth labels, a cleaned dataset of 1,876 mazes, a 29-dimensional per-cell feature

representation with a leakage-free split, and an exploratory analysis quantifying density fidelity, connectivity, and feature structure.

The next deliverables implement the remaining research steps: (i) training the feed-forward network to regress $h^*(n)/N$; (ii) integrating h_θ into A*; and (iii) evaluating nodes expanded, path-optimality ratio, and per-move latency against Manhattan A* and the exact oracle heuristic—thereby testing the project's central hypothesis.

References

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[2] E. W. Dijkstra, "A Note on Two Problems in Connexion with Graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.

[3] R. Yonetani, T. Taniai, M. Barekatain, and J. Fujimura, "Path Planning using Neural A* Search," in *Proc. International Conference on Machine Learning (ICML)*, 2021.

A Repository Structure

```
.
|-- README.md
|-- requirements.txt
|-- data_scripts/
|   |-- generate_mazes.py     # Phase 2: maze generation + generation_log.json
|   |-- compute_labels.py     # Phase 2: Dijkstra ground-truth labels
|   |-- clean_data.py         # Phase 2: cleaning (unreachable, dup, IQR)
|   |-- features.py           # Phase 2: 29-dim features + 70/15/15 split
|   `-- eda.py                # Phase 2: EDA figures
|-- figures/                  # EDA charts used in this report
|-- data/                     # generated data (git-ignored)
`-- report/
    |-- main.tex
    `-- main.pdf
```